SeedHammer II — the Wallet Policy journey

Engraving a wallet *someone else* built, and proving which one it is.

Three of this repo's journeys follow an operator who owns the seed. This one does not. The wallet arrives as a stack of cards; the operator's whole task is to satisfy themselves that the cards are *the right wallet* before committing them to steel — and the device has to make that possible without ever seeing a secret.

The policy is a depth-2 taproot script tree:

```
tr(@0/48'/0'/0'/2'/<0;1>/*,{{pk(@1/48'/0'/1'/2'/<0;1>/*),pk(@2/48'/0'/2'/2'/<0;1>/*)},pk(@3/48'/0'/3'/2'/<0;1>/*)})
```

Three properties of it are load-bearing. A taproot Merkle root depends on the tree's **shape**, so no flat descriptor can express this and the address can only be reached through the tree; the four cosigner keys are what push the card set to **8 md1 chunks**, making the gather a real multi-card gather rather than one tap; and four *distinct* keys mean a wrong key-to-slot mapping cannot accidentally produce the right address.

> **The keys are BIP-39's published test mnemonic** ("abandon … about") at `48'/0'/N'/2'`. Never put funds behind them.

What this wallet is, and is not

All four cosigners carry master fingerprint `73c5da0a`, because they are **accounts 0..3 of one seed**. That is *not* key reuse — four distinct xpubs, and the device's duplicate-key refusal has nothing to fire on — but it is a **single-seed** policy, which for a real four-cosigner wallet would defeat the point. It is here because it is the conformance corpus's wallet, and the corpus is what makes the comparison on the last page checkable against something other than one run of one script.

> **The first version of this journey declared the wrong origin for three of the four keys (F-217)**, and the fix is worth reading before the pages that follow. It passed `--path 48'/0'/0'/2'`, which *flattens* per-key origins to one shared path — so the card said all four keys live at account 0, when they live at accounts 0, 1, 2 and 3. Under BIP-32 a `(fingerprint, path)` pair names exactly *one* key, so that card described a wallet that cannot exist.

Nothing could have caught it downstream, and the re-run proves it: the four addresses below are byte-identical to the ones the broken card produced. Addresses derive from the xpubs a card *carries*, never from the origin it *declares*, so the device-vs-host comparison on the last page passed just as happily against the impossible card. Only a signer, asked to find its key, would ever have noticed.

The origins now live in the template — `@0/48'/0'/0'/2'/<0;1>/*` — which is where md has always taken per-key origins, and `md encode` refuses the flattened form outright. Nine of nine multi-key conformance vectors carried the same defect; all were regenerated.

1 · The host builds the card set

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(@0/48'/0'/0'/2'/<0;1>/*,{{pk(@1/48'/0'/1'/2'/<0;1
>/*),pk(@2/48'/0'/2'/2'/<0;1>/*)},pk(@3/48'/0'/3'/2'/<0;1>/*)}) --key @0=xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTex
cHZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUFL4r6KFrf --key @1=xpub6DzhyrnFFYQ1HimDiM388xHnDiRPNdZJFBmmxge3Y1WWcHLtMJ
LfRuhRHqnQCPbTj3fGKTuKFLHzzwpJkp5Dtc3UtLKZKaVZe1yqMBXd6Vk --key @2=xpub6EGx8sPr9FxPPE1rbZazhqWwpMXA3Hf5DYKtZbL7c4BSddzmQktp9
6UaTvecEkoCZysuaj79GMCFZYT1KKk7Ph2M3Kf5g8B82KZ8TZ9SKQR --key @3=xpub6E6Z3Ss5TXJYNJp4U1q3NZ3pCn82i7KXQAKUtNnzLJ3cCdchQeSdFvXe
mizaHUF7wNwRQAB8mPdoZhGHLiv49cWPtCnoJY3Az3E8JKxH9Mq --fingerprint @0=73c5da0a --fingerprint @1=73c5da0a --fingerprint @2=73c
5da0a --fingerprint @3=73c5da0a --force-chunked
chunk-set-id: 0xb5f80
md1fk huqrs 9q6tv yyy5j mpprj jtvyy 49ykc gfw2s qrqg3 g55j5 2cvgs 883w6 pfw0z a5z5u 796ay rwvv2 k4yg7 s
md1fk huqrs dq4h8 3w6pg tyzst hgx8e gtq4p cwl6u 5p2us 6r6zs nl2rd 0q6gg hvalg y07r0 ck54c n4ywn jh6nr r
md1fk huqrs 4mrqg drha0 m7uma pumfj 075dh zfzvy nh66n 94j5l cxlmx 9ayav 9mj0j jejcx y5wfr p8fs7 5nns9 s
md1fk huqrs lluyc agp8v v3nml gam2u g04zw 29zsq 0u7st 858ya cpa3h qfysj j3vzp kjnmf vwzwt hg79w xw0kr k
md1fk huqr3 xcs6h e5skl tczfz ylx0n dtm9f dvtp6 hyhac a0wl5 5lszs t97f5 wsrzq dxyxc qzsdm 9rkue zcref 9
md1fk huqr3 daj78 ussl5 37esf l5plj rjg55 pypuu 3qktl y4faz hvn9p hjf8p elz5w ehnrn xhgr7 3vdxs n7uez n
md1fk huqr3 jw8d5 9k03e u6d7x 7nza0 ynr9p 5d3ex an7jv up3kx 8742q 6d662 qzwwx 3qg6r mwyjl jawmu h6d4a c
md1fk huqr3 egdls 899kx 6k89k fkaz2 az67a jl8hy y967j e7sks 39z80 kl7qu z6axu vpqkr sg24z gatsl n
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

The operator carries 8 md1 chunks.
```

2 · …and, separately, what the device must prove to

These come from the *cards*, not from the arguments that produced them — `md inspect` reads the identity the engraved strings will actually carry.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fk huqrs 9q6tv yyy5j mpprj jtvyy 49ykc gfw2s qrq
g3 g55j5 2cvgs 883w6 pfw0z a5z5u 796ay rwvv2 k4yg7 s md1fk huqrs dq4h8 3w6pg tyzst hgx8e gtq4p cwl6u 5p2us 6r6zs nl2rd 0q6gg
hvalg y07r0 ck54c n4ywn jh6nr r md1fk huqrs 4mrqg drha0 m7uma pumfj 075dh zfzvy nh66n 94j5l cxlmx 9ayav 9mj0j jejcx y5wfr p8
fs7 5nns9 s md1fk huqrs lluyc agp8v v3nml gam2u g04zw 29zsq 0u7st 858ya cpa3h qfysj j3vzp kjnmf vwzwt hg79w xw0kr k md1fk hu
qr3 xcs6h e5skl tczfz ylx0n dtm9f dvtp6 hyhac a0wl5 5lszs t97f5 wsrzq dxyxc qzsdm 9rkue zcref 9 md1fk huqr3 daj78 ussl5 37es
f l5plj rjg55 pypuu 3qktl y4faz hvn9p hjf8p elz5w ehnrn xhgr7 3vdxs n7uez n md1fk huqr3 jw8d5 9k03e u6d7x 7nza0 ynr9p 5d3ex
an7jv up3kx 8742q 6d662 qzwwx 3qg6r mwyjl jawmu h6d4a c md1fk huqr3 egdls 899kx 6k89k fkaz2 az67a jl8hy y967j e7sks 39z80 kl
7qu z6axu vpqkr sg24z gatsl n
template: tr(@0/<0;1>/*,{{pk(@1/<0;1>/*),pk(@2/<0;1>/*)},pk(@3/<0;1>/*)})
n: 4
wallet-policy-mode: true
md1-encoding-id: b5f80796d9a3bbbc716ce5ffd7acc5a7
wallet-descriptor-template-id: 6f58bdf70d6f7d2fa82e8604bdb89317
wallet-policy-id: 4e67c6fd8220c32e51c9ad9947e24141
wallet-policy-id-fingerprint: 0x4e67c6fd
note: stdout is a keyless descriptor template (no keys)
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address --template tr(@0/48'/0'/0'/2'/<0;1>/*,{{pk(@1/48'/
0'/1'/2'/<0;1>/*),pk(@2/48'/0'/2'/2'/<0;1>/*)},pk(@3/48'/0'/3'/2'/<0;1>/*)}) --key @0=xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQw
CHbf5XB2mSTexcHZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUFL4r6KFrf --key @1=xpub6DzhyrnFFYQ1HimDiM388xHnDiRPNdZJFBmmx
ge3Y1WWcHLtMJLfRuhRHqnQCPbTj3fGKTuKFLHzzwpJkp5Dtc3UtLKZKaVZe1yqMBXd6Vk --key @2=xpub6EGx8sPr9FxPPE1rbZazhqWwpMXA3Hf5DYKtZbL7
c4BSddzmQktp96UaTvecEkoCZysuaj79GMCFZYT1KKk7Ph2M3Kf5g8B82KZ8TZ9SKQR --key @3=xpub6E6Z3Ss5TXJYNJp4U1q3NZ3pCn82i7KXQAKUtNnzLJ3
cCdchQeSdFvXemizaHUF7wNwRQAB8mPdoZhGHLiv49cWPtCnoJY3Az3E8JKxH9Mq --fingerprint @0=73c5da0a --fingerprint @1=73c5da0a --finge
rprint @2=73c5da0a --fingerprint @3=73c5da0a --count 2
bc1p2cycz66cyzltrvy5xd6v6g0jz3m5f9tjtz0aqkx9a5xet23su5rsa4mewu
bc1pa4sqhg0qw69zhqyaw3wv75ke49rwdf9cdc4taah3z5p58lp3zc8se4amnr
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address --template tr(@0/48'/0'/0'/2'/<0;1>/*,{{pk(@1/48'/
0'/1'/2'/<0;1>/*),pk(@2/48'/0'/2'/2'/<0;1>/*)},pk(@3/48'/0'/3'/2'/<0;1>/*)}) --key @0=xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQw
CHbf5XB2mSTexcHZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUFL4r6KFrf --key @1=xpub6DzhyrnFFYQ1HimDiM388xHnDiRPNdZJFBmmx
ge3Y1WWcHLtMJLfRuhRHqnQCPbTj3fGKTuKFLHzzwpJkp5Dtc3UtLKZKaVZe1yqMBXd6Vk --key @2=xpub6EGx8sPr9FxPPE1rbZazhqWwpMXA3Hf5DYKtZbL7
c4BSddzmQktp96UaTvecEkoCZysuaj79GMCFZYT1KKk7Ph2M3Kf5g8B82KZ8TZ9SKQR --key @3=xpub6E6Z3Ss5TXJYNJp4U1q3NZ3pCn82i7KXQAKUtNnzLJ3
cCdchQeSdFvXemizaHUF7wNwRQAB8mPdoZhGHLiv49cWPtCnoJY3Az3E8JKxH9Mq --fingerprint @0=73c5da0a --fingerprint @1=73c5da0a --finge
rprint @2=73c5da0a --fingerprint @3=73c5da0a --change --count 2
bc1pq67e92udm4tldq48jjdaravtekrw6k7u9apf6alhq2sfe8m9t6hsseycm5
bc1pf0lcpa0n7whukgcapvl022964qyu40h4cq7uc06puphjl28ntf3q9uck2h
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

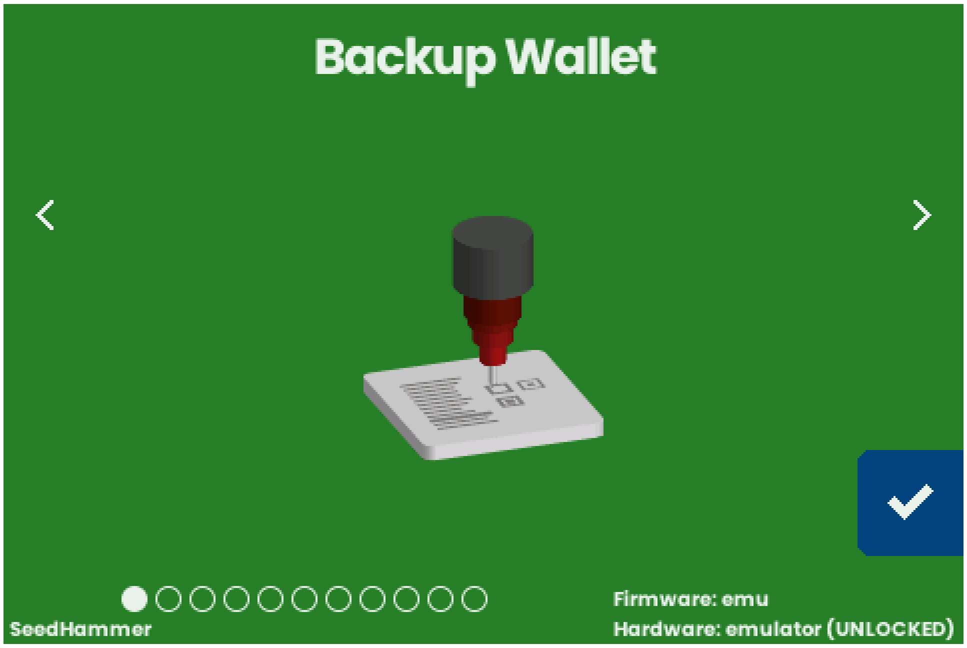
wallet id`wallet-descriptor-template-id: 6f58bdf70d6f7d2fa82e8604bdb89317` `wallet-policy-id: 4e67c6fd8220c32e51c9ad9947e24141`

receive 0..1`bc1p2cycz66cyzltrvy5xd6v6g0jz3m5f9tjtz0aqkx9a5xet23su5rsa4mewu` `bc1pa4sqhg0qw69zhqyaw3wv75ke49rwdf9cdc4taah3z5p58lp3zc8se4amnr`

change 0..1`bc1pq67e92udm4tldq48jjdaravtekrw6k7u9apf6alhq2sfe8m9t6hsseycm5` `bc1pf0lcpa0n7whukgcapvl022964qyu40h4cq7uc06puphjl28ntf3q9uck2h`

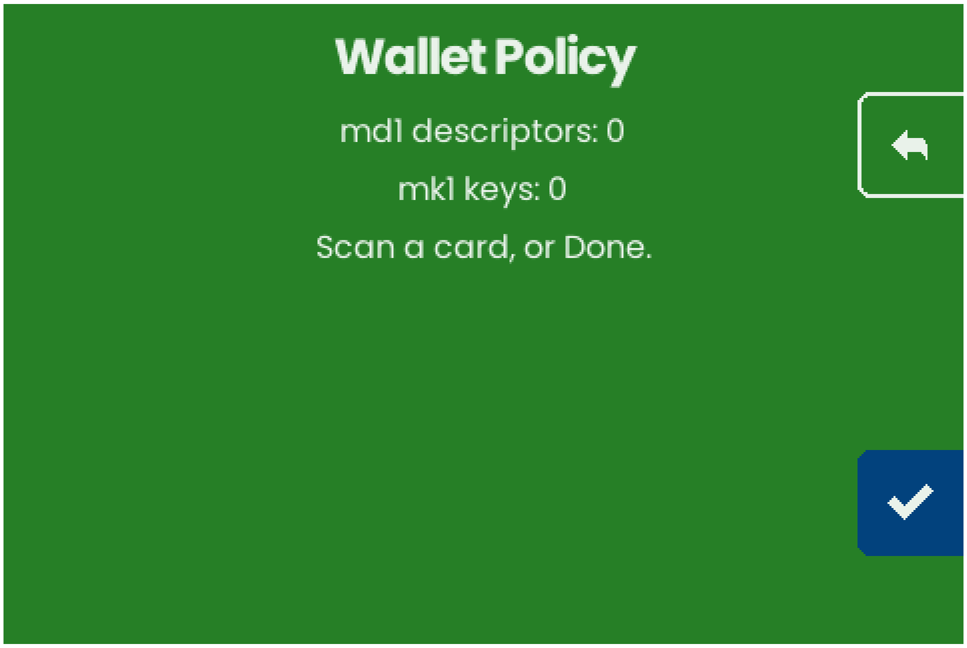
The fingerprints are part of the identity. Measured while building this journey: the same template, the same four xpubs and the same origin, encoded *without* `--fingerprint`, gives a 7-chunk set whose wallet-policy-id is `0b3b95a4`… instead of `ade5967c`…. The template id is unchanged either way, because it is key-stable. An operator who omits them gets a card that proves to a different id than their coordinator shows.

3 · The device

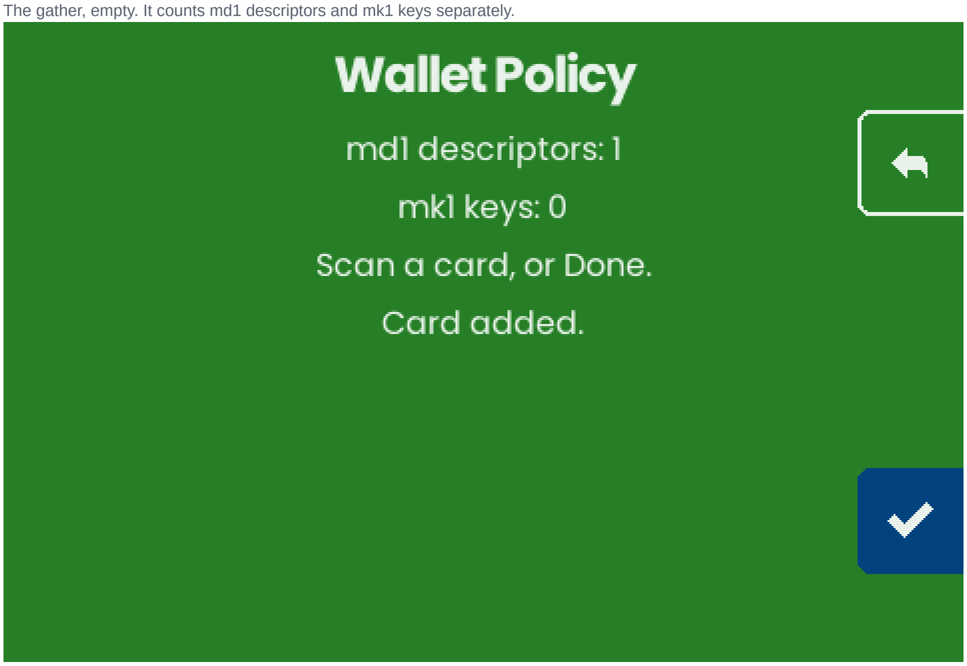


Home. The emulator ships a systemwide payload, so the walk skips its LOAD/SKIP prompt first — this operator's wallet arrives on cards.

Wallet Policy, the eighth entry — beside the other engrave programs, not after the utilities.



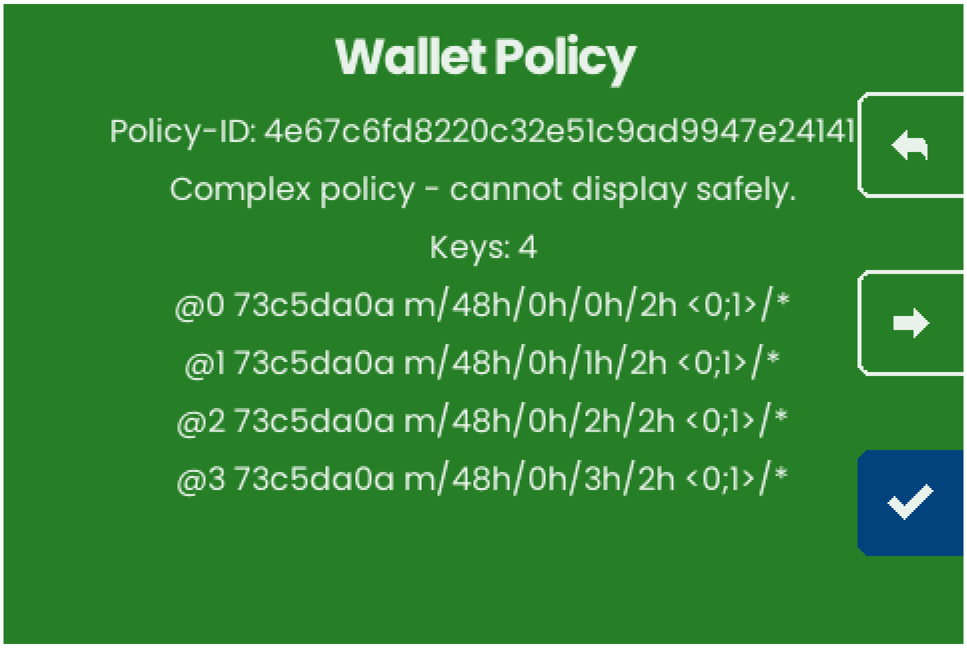
The gather, empty. It counts md1 descriptors and mk1 keys separately.



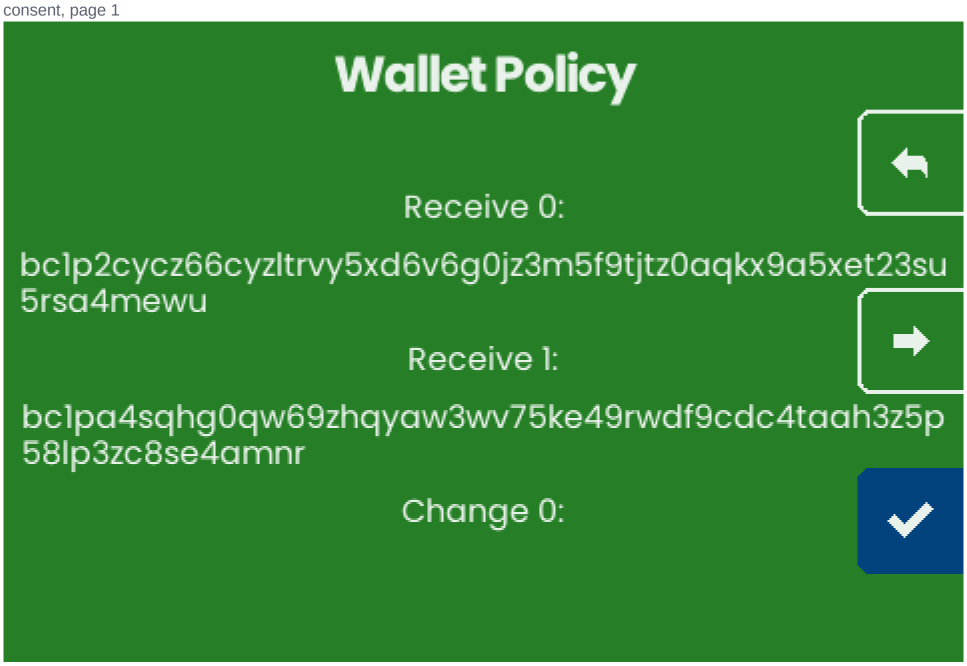
After all 8 chunks cross the reader: ONE card. The tally counts cards, not chunks — a card's intermediate chunks draw nothing, and a dropped one means the set never completes.

4 · The proof, on the consent path

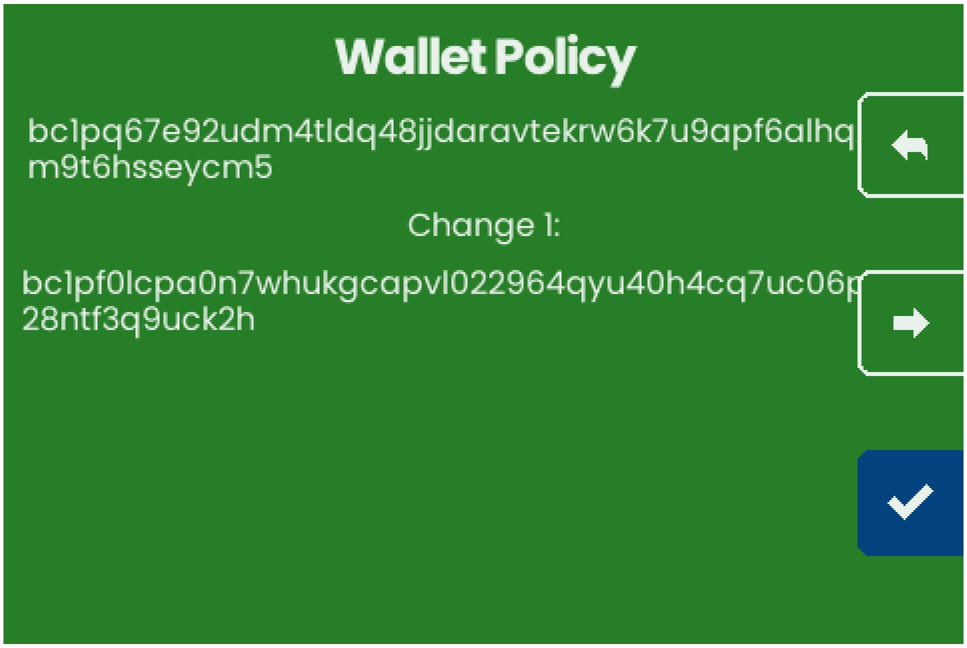
Not beside it. The addresses are lines on the screen the operator has to pass through to engrave, so tapping straight on still shows them.



consent, page 1



consent, page 2



consent, page 3

The id is **named**. Two exist for one wallet — the key-*stable* Template-ID and the key-*dependent* Policy-ID — and they are indistinguishable once rendered as 16 bytes of hex. An operator comparing the wrong one against their coordinator sees a mismatch and reads it as a corrupted backup, so the screen says which it is showing.

5 · What this run actually established

The capture is a **comparison, not a photo shoot**. `shots_walletpolicy.js` is handed the host's id and addresses and throws if the screen disagrees, so these pages cannot show agreement that did not happen.

chunks presented8
cards gathered1
consent pages read3
wallet id matched`4e67c6fd8220c32e51c9ad9947e24141`
addresses matched`bc1p2cycz66cyzltrvy5xd6v6g0jz3m5f9tjtz0aqkx9a5xet23su5rsa4mewu`
`bc1pa4sqhg0qw69zhqyaw3wv75ke49rwdf9cdc4taah3z5p58lp3zc8se4amnr`
`bc1pq67e92udm4tldq48jjdaravtekrw6k7u9apf6alhq2sfe8m9t6hsseycm5`
`bc1pf0lcpa0n7whukgcapvl022964qyu40h4cq7uc06puphjl28ntf3q9uck2h`

Two implementations, in two languages, from the same four xpubs: Rust derived these off-device and the firmware derived them again from the cards, and they are the same strings. The negative control was run — corrupting one character of one expected address fails the capture — so the match is a measurement rather than a coincidence of both sides being empty.

Reproducing

```
bash transcript_walletpolicy.sh > transcript_walletpolicy.txt 2>&1
python3 capture_walletpolicy.py     # rebuilds emu.wasm, drives it, writes shots/
python3 build_pdf_walletpolicy.py   # this document
```

What this journey does NOT show

- **No engraving.** The walk stops at consent. Cutting is the Engrave Bundle path, already covered by the pathological journey, and the new thing here is the proof that precedes it.
- **No mk1 key cards.** The program accepts them, and a keyless template gathered alongside its key plates still reports no addresses — that is F-216, deliberately unbuilt because mapping an mk1 to an `@N` slot has no obvious rule and a wrong mapping would show a *wrong* address as proof.
- **Not every shape.** A tap leaf outside `pk`/`multi_a`/`sortedmulti_a` is refused rather than approximated (F-214).